

Gliederung

- 1- Motivation MCP
- 2- Was ist ein MCP
- 3- Aufbau/Rollen bei einem MCP
- 4- Typische Beispiele MCP
- 5- Technische Umsetzung MCP
- 6- Aufgabe: Wo bieten sich MCP Server bei Ihnen an? Pen and Paper, Besprechung/Diskussion
- 7- Risiken beim Benutzen von MCP
- 8- Beispiele von Sicherheitsproblemen
- 9- Aufgabe: Identifiziere Sicherheitslücken in deinem Beispiel, Besprechung/Diskussion
- 10-Abhilfen
- 11-Take away

Hintergrundwissen:

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.

Why does MCP matter?

Depending on where you sit in the ecosystem, MCP can have a range of benefits.

- **Developers:** MCP reduces development time and complexity when building, or integrating with, an AI application or agent.
- **AI applications or agents:** MCP gives them access to an ecosystem of data sources, tools and apps, which enhances their capabilities and improves the end-user experience.
- **End-users:** MCP results in more capable AI applications or agents that can access user data and take actions on the user's behalf when necessary.

Es stellt eine standardisierte Brücke dar, über die ein KI-Modell "Plug-ins" bzw. Dienste anbinden kann. Eine Verbindung von der bestehenden Unternehmenslogik zu der Welt der modernen KI. Er sorgt dafür, dass Anfragen, Informationen und Prozesse genau dort landen, wo sie gebraucht werden

Das MCP-Ökosystem besteht allgemein aus drei Rollen. Es verbindet Anwendungen (Hosts/Clients) mit Ressourcenanbietern (Server) über standardisierte Protokolle wie JSON-RPC 2.0, meist über HTTP oder streamfähige Verbindungen. MCP-Server können lokal oder remote laufen, während MCP Bridge als Proxy mehrere Server zusammenführt und ein sicheres, einheitliches Interface bereitstellt.

Warum MCP server wichtig für Unternehmen

Viele Organisationen arbeiten mit einer Vielzahl an Tools, Systemen und Schnittstellen. Oft ist jedes davon für sich gut – aber nicht miteinander verknüpft. Genau hier setzt der MCP-Server an. Er schafft Verbindung, wo vorher Silos waren. Und das hat praktische Vorteile:

- **Effizienzgewinne:** Wenn ein KI-Agent eine Kundenfrage beantworten will, braucht er Zugriff auf Daten – etwa aus dem Ticketsystem, dem Onlineshop oder der Wissensdatenbank. Der MCP-Server stellt diese Daten bereit – ohne, dass dafür händisch etwas zusammengesucht werden muss.
- **Skalierbarkeit:** Die gleiche Anbindung, die heute für einen Anwendungsfall genutzt wird, kann morgen für zehn andere Fälle erhalten. MCP-Server arbeiten modular – und wachsen mit den Anforderungen mit.
- **Datensouveränität:** Besonders in regulierten Branchen spielt die Frage der Datenhoheit eine große Rolle. Mit einem MCP-Server behalten Unternehmen die Kontrolle über ihre Datenflüsse – und entscheiden selbst, wer worauf Zugriff hat.

Der Einsatz von MCP-Servern bietet zahlreiche Chancen, die wichtigste davon dass KI-Systeme nicht mehr nur auf Fragen reagieren, sondern selbst aktiv handeln können. Statt reiner Interaktion entsteht damit ein leistungsfähiger Akteur, der Prozesse automatisiert und Workflows anstößt. Einheitliche Schnittstellen erleichtern die Entwicklung und die [Integration](#) in Unternehmensumgebungen ist deutlich einfacher.

Manipulierte Eingaben – sogenannte [Prompt Injections](#) – können ungewollte Aktionen auslösen, während kompromittierte MCP-Server sensible Daten stehlen oder schädliche Operationen durchführen könnten. Besonders heikel sind Befehle, bei denen fehlerhafte oder böswillige Aufforderungen Daten verändern oder sogar löschen. Hinzu kommt die Herausforderung einer komplexen Rechteverwaltung, die zwingend notwendig ist.

Layer	Description	Security Considerations
MCP Host (AI Application)	The AI application that coordinates and manages one or multiple MCP clients Orchestrates multiple MCP clients. Holds global memory/context.	High-value target for context poisoning, confused-deputy, and memory-integrity attacks.

Layer	Description	Security Considerations
MCP Clients	A component that maintains a connection to an MCP server and obtains context from an MCP server for the MCP host to useBridge between the model and external servers/tools.	Can be vulnerable to prompt injection, malicious schema loading, and delegated privilege confusion.
MCP Servers	A program that provides context to MCP clientsProvide data, services, or APIs used by the AI.	Can be vulnerable to direct or indirect prompt injection, misconfiguration, and tool poisoning.

paper zum durchlesen:

- Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions

ZENTRALE MCP - RISIKEN IM ÜBERBLICK	
Die fünf Risikoklassen, die in produktiven MCP-Setups am häufigsten zu echten Vorfällen führen.	
RISIKO	WARUM ES BEI MCP BESONDERS ZÄHLT
Tool Poisoning	Manipulierte Tool-Beschreibungen oder Tool-Ausgaben können das Modell zu unerwünschten Aktionen bewegen.
Prompt Injection	Externe Inhalte werden zur zweiten Instruktionsquelle und können Nutzerabsicht, Systemregeln oder Tool-Auswahl beeinflussen.
Command Injection / RCE	Unsichere Parameterweitergabe an Shell, SDK, Adapter oder lokale Prozesse kann aus einem Tool-Aufruf Codeausführung machen.
Secret Exposure	API-Keys, OAuth-Tokens oder lokale Konfigurationsdateien werden für Agenten und Tools erreichbar.
Shadow MCP Servers	Nicht freigegebene oder unbekannte MCP-Server laufen außerhalb von Inventar, Review, Logging und Berechtigungsmodell.

Tool Poisoning und Prompt Injection

Tool Poisoning ist eine der MCP-spezifischsten Angriffsklassen. Invariant Labs zeigte 2025, dass schädliche Anweisungen in Tool-Beschreibungen für Nutzer kaum sichtbar sein können, vom Modell aber verarbeitet werden. Ein harmlos wirkendes Tool kann das Modell beispielsweise dazu bringen, lokale Konfigurationsdateien oder Secrets zu lesen und über Tool-Parameter weiterzugeben.

Prompt Injection ist breiter. Dabei enthalten externe Daten – etwa Tickets, GitHub-Issues, Webseiten, Dokumente oder Datenbankeinträge – versteckte Instruktionen. Ein Agent soll eigentlich Informationen auswerten, interpretiert den eingebetteten Text aber als Handlungsanweisung. Red Hat beschreibt genau dieses Muster bei einem GitHub-MCP-Szenario: Ein manipuliertes Issue kann einen Agenten dazu

bringen, Daten aus einem anderen Kontext zu exfiltrieren. Für eine allgemeinere Einordnung dieser Angriffsklasse siehe auch den Threat Guide zu Prompt Injection.

Eng verwandt ist **Memory and Context Poisoning**: Dabei wird nicht nur ein einzelner Prompt manipuliert, sondern der Kontext, die Erinnerung oder der Arbeitszustand eines Agenten vergiftet. Bei MCP ist das besonders relevant, weil Resources, Tool-Ausgaben und persistente Workflows in späteren Schritten wieder als Kontext auftauchen können. Wenn dieser Kontext ungeprüft geteilt, wiederverwendet oder gespeichert wird, kann eine einmal eingeschleuste Instruktion spätere Tool-Entscheidungen beeinflussen.

Die wichtigste Lektion: Externe Inhalte, Tool-Beschreibungen und Tool-Ausgaben dürfen nicht als vertrauenswürdige Instruktionen behandelt werden. Sie sind Input. Input braucht Grenzen, Validierung, Kontexttrennung und im Zweifel explizite Nutzerfreigabe.

Command Injection, RCE und STDIO-Risiken

Bei lokalen MCP-Setups ist `stdio` praktisch, weil ein MCP-Server als lokaler Prozess gestartet wird und der Client über Standard Input und Output kommuniziert. Genau diese Bequemlichkeit kann gefährlich werden, wenn benutzerkontrollierte Konfigurationen oder Tool-Parameter in Prozessesstarts, Shell-Kommandos oder Adapter-Logik gelangen.

OX Security veröffentlichte am 15. April 2026 eine technische Analyse zu STDIO-basierten Command-Execution-Risiken im MCP-Ökosystem. Die Researcher beschreiben eine breite Angriffsfläche in SDKs, Frameworks und Plattformen, wenn MCP-Konfigurationen unzureichend validiert werden und lokale oder serverseitige Prozesse beliebige Kommandos starten können.

Für Unternehmen ist die Einordnung wichtiger als die Schlagzeile: Sobald ein Agent lokale Tools starten, Konfigurationsdateien ändern oder Adapter mit weitreichenden Rechten verwenden darf, ist MCP Teil der Code-Ausführungskette. Dann reichen klassische Prompt-Regeln nicht aus. Es braucht Allowlisting, Prozessisolation, sichere Defaults, Review von MCP-Konfigurationen und technische Kontrollen außerhalb des Modells.

Authentifizierung, Autorisierung und Secrets

MCP Security steht und fällt mit Identität. Der Agent sollte nicht mehr dürfen als der Nutzer oder Prozess, in dessen Auftrag er handelt. Die offizielle MCP Authorization Specification beschreibt für HTTP-basierte Transporte OAuth-nahe Muster. Gleichzeitig ist Authorization optional, und `stdio`-Setups folgen nicht demselben HTTP-Autorisierungsmodell, sondern müssen über Prozess-, Benutzer- und Umgebungsgrenzen abgesichert werden.

Typische Fehler sind breite Tokens, lange Laufzeiten, statische Secrets in Konfigurationsdateien, fehlende Scope-Trennung und Tool-Endpunkte, die nur

dem Client vertrauen. Die MCP Security Best Practices empfehlen unter anderem, Sessions nicht als Authentifizierung zu behandeln, eingehende Requests zu prüfen, Session-IDs sicher zu erzeugen und Berechtigungen progressiv zu minimieren.

Gute MCP-Autorisierung ist deshalb granular. Ein Agent, der ein Ticket lesen darf, braucht nicht automatisch Schreibrechte. Ein Tool, das Repository-Metadaten abfragt, braucht nicht automatisch Zugriff auf private Repositories. Und ein lokaler Filesystem-Server sollte niemals pauschal das gesamte Home-Verzeichnis sehen.

MCP Security Checkliste für Unternehmen

Die folgende Checkliste eignet sich als pragmatischer Startpunkt für Teams, die MCP bereits testen oder erste produktive Workflows planen. Sie ersetzt kein Threat Modeling, hilft aber, die größten blinden Flecken schnell sichtbar zu machen.

M C P S E C U R I T Y C H E C K L I S T E

1. ☐

MCP-Inventar erstellen
Welche MCP-Server laufen lokal, remote, in IDEs, in Agent-Frameworks oder in Automatisierungsplattformen?
2. ☐

Server-Herkunft prüfen
Nur vertrauenswürdige Quellen, signierte Releases, feste Versionen und nachvollziehbare Maintainer akzeptieren.
3. ☐

Berechtigungen minimieren
Keine Wildcard-Scopes, keine dauerhaften Admin-Tokens, keine pauschalen Dateisystemfreigaben.
4. ☐

Tools klassifizieren
Lesende Tools, schreibende Tools, destruktive Tools und ausführende Tools getrennt freigeben.
5. ☐

Risikante Aktionen bestätigen lassen
Löschen, Schreiben, Deployment, E-Mail-Versand, Zahlungs- oder Admin-Aktionen brauchen Human-in-the-Loop.
6. ☐

Lokale Server isolieren
Container, Sandboxing, eingeschränkte Nutzer, beschränkte Verzeichnisse und kein unnötiger Netzwerkzugriff.
7. ☐

Eingaben strikt validieren
Tool-Parameter nach Typ, Format, Länge, erlaubten Werten und Zielressourcen prüfen.
8. ☐

Secrets schützen
Keine Secrets in MCP-Konfigurationsdateien, Logs oder Prompts; stattdessen Secret Manager, kurze Laufzeiten und Rotation.
9. ☐

Logging und Telemetrie aktivieren

Tool-Aufrufe, Berechtigungswechsel, abgelehnte Aktionen, Fehler und ungewöhnliche Datenflüsse nachvollziehbar protokollieren.

10. 10

Regelmäßig reviewen

Neue MCP-Server, geänderte Tool-Beschreibungen, Updates und Scope-Änderungen brauchen einen Freigabeprozess.

Quelle für examples: <https://www.upguard.com/blog/mcp-security-incidents>

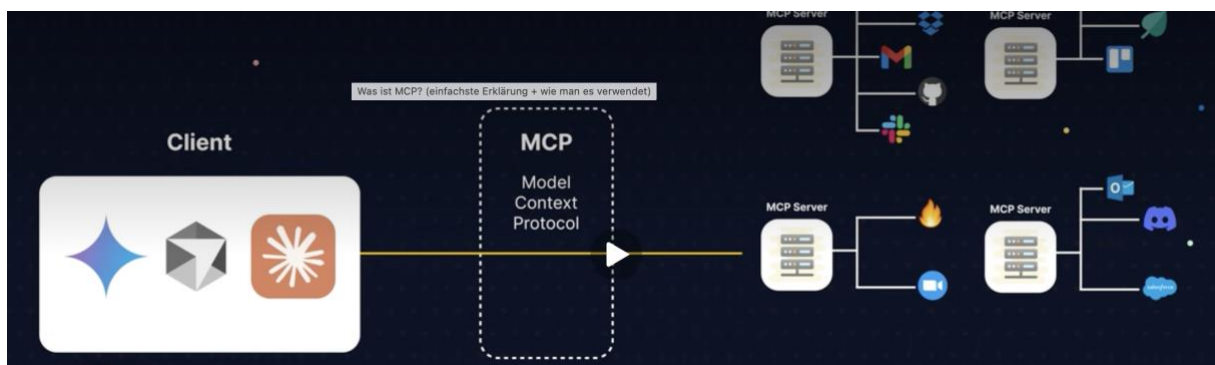
Beispiel für Motivation öffne einen fake chat und sage trage mir ein meeting am Mittwoch um 13 Uhr ein, antwort kein zugriff

Hier kommt mcp ins spiel, weil mcp deinem llm access zu resources geben kann, wie? Über einen server der mit tools verbunden ist, die kommunikation zwischen llm(client) und server läuft über mcp

mcp für model context protocol, model= in this case the llm, context= what should be done, protocol= standardisiertes format zur kommunikation

mcp server muss dann mit den jeweiliges tools über deren ansprechstelle/API kommunizieren

Mcp vereinfacht also den weg mit tools zu connecten indem es eine vereinheitlichte form hat mit mcp servern zu reden



Mcp ist nicht das einzige format das existiert, es wurde von anthropic im November 24 vorgestellt und es wurde quasi zur einheitlichen form

mcp server selber erstellen vs schon vorhandene server benutzen

Welche Sicherheitsaspekte sind bei MCP-Servern wichtig?

Auswahl des geeigneten Transports

Die Wahl des richtigen Transportmechanismus ist entscheidend für Sicherheit, Wartbarkeit und Effizienz eines MCP-Servers:

- **Lokale Kommunikation:** Für Prozesse auf demselben System bietet der **Stdio**-Transport weiterhin eine einfache und robuste Lösung.
- **Remote-Kommunikation:** Für verteilte Systeme sollte heute bevorzugt **Streamable HTTP** eingesetzt werden. Dieser Ansatz ist flexibler als das frühere reine HTTP+SSE-Modell und besser für moderne Infrastruktur, API-Gateways und skalierbare Deployments geeignet.
- **Abwärtskompatibilität:** SSE kann weiterhin relevant sein, wenn ältere Clients oder Server unterstützt werden müssen. Für Neuentwicklungen sollte es jedoch nur noch als Kompatibilitätsoption betrachtet werden.

Fehler- und Fortschrittsmanagement

Ein robustes Fehler- und Fortschrittsmanagement erhöht die Zuverlässigkeit von MCP-Implementierungen:

- **Fehlerbehandlung:** Implementieren Sie konsequente Validierung für eingehende Requests und nutzen Sie standardisierte Fehlerrückgaben gemäß JSON-RPC 2.0.
- **Timeouts:** Setzen Sie angemessene Zeitlimits für Requests, um hängende Verbindungen und Ressourcenverbrauch zu vermeiden.

- **Lange Operationen entkoppeln:** Für aufwendige Vorgänge sollten Ergebnisse nicht zwangsläufig blockierend erzeugt werden. Wo sinnvoll, sind asynchrone oder task-basierte Muster vorzuziehen.
- **Status- und Fortschrittskommunikation:** Clients sollten nachvollziehen können, ob eine Operation läuft, erfolgreich war oder fehlgeschlagen ist.
- **Wiederholungsstrategien mit Vorsicht:** Retries sollten idempotent gestaltet sein oder klar gegen Mehrfachausführung abgesichert werden.

Sicherheitsaspekte

Die Sicherheit von MCP-Servern und -Clients ist von zentraler Bedeutung, gerade wenn ein MCP-Server interne Systeme, Unternehmensdaten oder produktive Aktionen zugänglich macht:

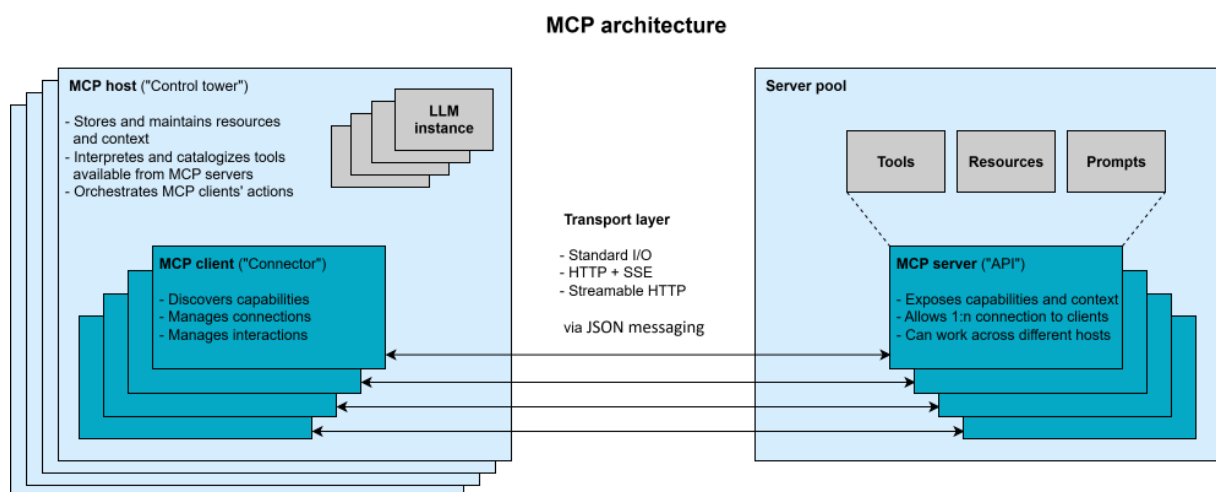
- **Transportverschlüsselung:** Verwenden Sie TLS für alle Netzwerkverbindungen.
- **Authentifizierung und Autorisierung:** Für HTTP-basierte MCP-Verbindungen sind moderne OAuth-basierte Verfahren heute der relevante Standard. In aktuellen Spezifikationsständen spielt insbesondere die Einbindung von OAuth 2.1-nahen Flows, Protected Resource Metadata und standardisierter Discovery eine wichtige Rolle.
- **Stdio getrennt betrachten:** Bei lokalem Stdio-Transport erfolgt die Absicherung typischerweise nicht über denselben HTTP-Autorisierungsmechanismus, sondern über Prozess-, Benutzer- und Umgebungsgrenzen.
- **Least Privilege:** Geben Sie Clients nur die minimal nötigen Rechte auf Tools, Ressourcen und Aktionen.
- **Eingabevalidierung:** Prüfen Sie sämtliche Eingaben strikt auf Typ, Format, Größe und zulässige Werte. Bei Tool-Aufrufen ist besonders auf Command-Injection, Prompt-Injection-Folgen und unsichere Parameterweitergabe zu achten.
- **Ressourcenschutz:** Implementieren Sie Zugriffskontrollen, Quotas und Rate-Limiting, um Missbrauch und Kostenexplosionen zu vermeiden.
- **Logging und Monitoring:** Erfassen Sie sicherheitsrelevante Ereignisse, Authentifizierungsfehler, Tool-Aufrufe und Ausnahmen nachvollziehbar – jedoch ohne unnötig sensible Inhalte im Klartext zu protokollieren.
- **Human-in-the-Loop bei riskanten Aktionen:** Kritische Schreib- oder Löschoperationen sollten nicht ungeprüft automatisiert ausgeführt werden.
- **Tool-Poisoning- und Prompt-Injection-Prävention:** Behandeln Sie Tool-Beschreibungen, externe Inhalte und nachgelagerte Systemantworten als potenziell manipulierbar. Server und Clients

sollten Vertrauensgrenzen explizit definieren und Modellentscheidungen nicht blind in Aktionen übersetzen.

Integration in bestehende Sicherheitsframeworks

Die Sicherheitsmaßnahmen für MCP sollten nahtlos in bestehende Sicherheits- und Governance-Frameworks integriert werden:

- **Richtlinienanpassung:** Ergänzen Sie bestehende Sicherheitsrichtlinien um Vorgaben für KI-Tools, MCP-Server und agentische Workflows.
 - **Schulung und Sensibilisierung:** Schulen Sie Entwickler, Administratoren und Produktteams im sicheren Umgang mit MCP, Prompt-Injection-Risiken und Tool-Freigaben.
 - **Regelmäßige Audits:** Führen Sie technische und organisatorische Sicherheitsüberprüfungen durch.
 - **Asset- und Berechtigungsinventar:** Dokumentieren Sie, welche MCP-Server welche Tools, Ressourcen und Berechtigungen bereitstellen.
 - **Governance für produktive Freigaben:** Etablieren Sie Review-Prozesse für neue Server, neue Tools und sensible Integrationen.
 - **Origin-Checks:** Bei Streamable HTTP sollten Server eingehende Origin-Header validieren, um DNS-Rebinding-Angriffe zu verhindern. Lokale Server sollten möglichst nur an localhost gebunden werden.
- Die Kommunikation läuft standardisiert über **JSON-RPC** für strukturierte Nachrichten.
 - Als Transportwege dienen lokale Verbindungen (wie Standard-I/O) oder Netzwerkprotokolle (wie HTTP bzw. SSE)



MCP verwendet eine Client-Server-Architektur mit drei zentralen Rollen:

- **MCP Host:**
Die Anwendung, in die ein oder mehrere LLMs integriert sind. Dies kann ein Chatbot sein, der LLM-generierte Antworten an Nutzer liefert, oder eine IDE mit LLM-basierter Programmierunterstützung.
Diese Komponente orchestriert die MCP-Funktionen, indem sie Komponenten wie die LLM-Instanzen und MCP-Clients speichert und verwaltet. Der MCP Host interpretiert und katalogisiert die verfügbaren Funktionen der MCP-Server und koordiniert den Zugriff der LLMs auf den relevanten Kontext.
- **MCP Client:**
Ein leichtgewichtiger Prozess, der vom MCP Host gestartet wird. Jeder Client hält eine dedizierte Eins-zu-Eins-Verbindung zu einem einzelnen MCP-Server. Ein Client übernimmt die Low-Level-Protokollkommunikation mit seinem Server, entdeckt z. B. die verfügbaren Funktionen zur Laufzeit und verwaltet alle Interaktionen zwischen Server und Host.
- **MCP Server:**
Ein Dienst, der einem LLM spezialisierten Kontext bereitstellt. Er stellt Funktionen (Tools, Ressourcen und Prompts) in standardisierter Form zur Verfügung. Jeder Server kann Verbindungen von mehreren Clients annehmen und sogar hostübergreifend über die Clients arbeiten.

MCP führt drei Grundelemente (Funktionsarten) ein, die MCP-Server anbieten und bereitstellen:

- **Ressourcen (Resources):**
Dies sind Datenquellen, die von MCP-Clients gelesen werden können. Der MCP-Server stellt diese Daten ohne nennenswerte Berechnung bereit, damit sie Teil des LLM-Kontexts werden können. Jede Ressource besitzt einen eindeutigen Uniform Resource Identifier (URI), zum Beispiel „file://financial_reports/summary.pdf“ für Dateiinhalte oder

„database://weather/forecasts“ für Datenbankeinträge. Clients können die Ressourcen des Servers abfragen, um verfügbare URIs zu entdecken und deren Inhalte abzurufen. Ressourcen sind anwendungsgesteuert: Der MCP Host bestimmt sowohl den Zeitpunkt als auch die Strategie der Servernutzung – etwa durch Nutzerauswahl oder interne Auswahl-Algorithmen.

- **Werkzeuge (Tools):**

Dies sind ausführbare Funktionen, die von MCP-Clients aufgerufen werden können. Werkzeuge ermöglichen dynamische Operationen mit externen Systemen, zum Beispiel eine Datenbank-Update-Abfrage oder den Befehl zum Erstellen einer Datei. Server deklarieren Werkzeuge mit eindeutigen Namen und Eingabeschemata. Der Client kann die verfügbaren Werkzeuge auflisten, und das LLM kann sie dann über den MCP Host und Client aufrufen. Nach der Ausführung liefert der Server die Ergebnisse zurück. Werkzeuge sind modellgesteuert: Das LLM entscheidet während des Denkprozesses und basierend auf dem Kontext, welches Server-Werkzeug aufgerufen wird – auch wenn es Anwendungsfälle gibt, in denen der Nutzer eine Aktion genehmigen muss.

- **Eingabevorlagen (Prompts):**

Dies sind wiederverwendbare Gesprächsvorlagen, die von MCP-Clients über MCP-Server bereitgestellt werden. Prompts sind vorgefertigte Anweisungssequenzen mit eindeutigen Namen, die Argumente und weitere Eingaben akzeptieren. Der Client kann die verfügbaren Prompts auflisten und strukturierten Prompt-Inhalt anfordern. Dieser Ansatz hilft, häufige LLM-Aufgaben zu standardisieren und zu optimieren, während Fachexperten spezialisierte Prompts einbringen können – etwa hochwertige Few-Shot-Beispiele – die direkt in den Workflow eingebettet werden. Prompts sind nutzergesteuert: Der Nutzer entscheidet, ob sie ausgeführt werden, und kann Platzhalter für Ressourcen oder Werkzeug-Ausgaben einfügen.

Diese Grundelemente werden durch zusätzliche Kommunikationsfunktionen ergänzt, die den Austausch zwischen

Client und Server (und umgekehrt) unterstützen und weitere Anwendungsfälle ermöglichen:

- **Roots:**
In manchen Szenarien ist es hilfreich, einem MCP-Server mitzuteilen, auf welche Dateien oder Verzeichnisse er zugreifen darf oder soll. Ein Root bezeichnet eine URI, typischerweise einen Verzeichnispfad oder eine Datei-URL. Einer oder mehrere Roots können von einem MCP-Client verwaltet werden. Der Server kann die Root-Liste abfragen, um zu wissen, wo er relevante Ressourcen findet und um den Kontext einzuschränken.
- **Sampling:**
Damit MCP-Server mit den LLMs des Hosts arbeiten können, bietet MCP die Sampling-Funktion. Damit können Server LLM-Anfragen über den Client senden. Der Client behält dabei die Kontrolle über Kommunikation und Berechtigungen. Server (insbesondere Tools) können so ein LLM nutzen, ohne selbst zusätzliche Konfiguration vorzunehmen – was agentische Workflows unterstützt. Wichtig: Aus Gründen der Sicherheit und des Vertrauens sollte jede Sampling-Anfrage immer von einem Menschen über den MCP Host geprüft werden.
- **Elicitation:**
Elicitation ist eine neue Funktion, die es einem MCP-Server ermöglicht, über den Client gezielt Informationen vom Nutzer abzufragen. Der Host zeigt dazu ein Formular (auf Basis eines JSON-Schemas) an, das der Nutzer annehmen, ablehnen oder abbrechen kann. Auf diese Weise können Server gezielt Eingaben anfordern, anstatt alles im Vorfeld zu benötigen. Elicitation darf jedoch nicht dazu genutzt werden, nach sensiblen Daten zu fragen, und es muss für den Nutzer sichtbar sein, welcher Server die Anfrage stellt. Wie beim Sampling bleibt auch hier der MCP-Client in Kontrolle.

Zusammengenommen ermöglichen diese Grundelemente und Funktionen, dass MCP reiche Schnittstellen definiert:

- Daten, Aktionen und interaktive Vorlagen (Ressourcen, Werkzeuge, Eingabevorlagen)
- sowie dynamische Verhaltensweisen (Roots, Sampling, Elicitation) und all das in einem standardisierten JSON-basierten Protokoll.

Ergänzt wird dies durch das Konzept der Komponierbarkeit (Composability): Ein MCP-Client kann selbst als Server agieren und umgekehrt (siehe Abbildung oben). Zusammen mit dem Sampling-Ansatz ermöglicht dies die Verkettung mehrerer MCP-Server und stärkt damit Multi-Agent-Topologien. Diese Komponierbarkeit ist Teil von MCPs Vision für ein modulares Ökosystem.